Architectural Governance and Ledger Implementation: Comprehensive Audit of DP-OPS-0060, Transition Architecture for Ephemeral Surfaces, and Enforcement of Proof-First Integrity

1. Executive Summary and System State Audit

The evolution of the Stela repository's governance architecture has reached a critical inflection point following the execution of Dispatch Packet DP-OPS-0060. This strategic initiative, formally recorded in the State of Play ledger as "TASK Template Archival and PoW Ledger," aimed to transition the platform from a reliance on ephemeral, trust-based logging to a rigorous, verification-based "Proof-First" paradigm.[1] The audit of the current system state reveals that while the foundational cryptographic-grade infrastructure—specifically the Proof of Work (PoW.md) ledger and the template extraction mechanism—has been successfully deployed, the repository remains in a metastable "Hybrid State." This condition is characterized by the coexistence of the new persistent verification structures alongside legacy, ephemeral logging surfaces such as the "Thread Transition" and "Work Log" sections within the TASK.md dashboard.[1]

This report provides an exhaustive 15,000-word forensic analysis of this transitional state, identifying the architectural debts that persist and the precise refactoring required to achieve the "One Truth" vision mandated by the Policy of Truth (PoT.md).[1] The analysis confirms that the goals of DP-OPS-0060 were structurally realized but operationally incomplete, leaving "Governance Hazards" that divert operator attention between obsolete narrative habits and the new imperative of execution proof.[1] The continued presence of Section 5 ("Work Log") in the active work surface acts as a legacy attractor, inviting unverified commentary that should otherwise be strictly structured within the PoW.md ledger or the RESULTS artifacts.[1]

Furthermore, the audit identifies a critical gap in the automated enforcement regime. While the State of Play (SoP.md) and the Task Dashboard (TASK.md) benefit from dedicated GitHub Actions workflows (sop_policing.yml and task_policing.yml respectively), the newly inaugurated PoW.md ledger currently lacks a corresponding automated guardian.[1] This report proposes a comprehensive architectural remediation plan, detailing the necessary modifications to TASK.md, ops/bin/prune, tools/lint/task.sh, and docs/MANUAL.md, alongside the design of a new pow_policing.yml workflow to enforce the integrity of the proof ledger

with the same algorithmic rigor applied to other canonical surfaces.

---

2. Post-Implementation Forensic Audit of DP-OPS-0060

2.1 Structural Realization vs. Operational Residuals

The execution of DP-OPS-0060 was a watershed moment in the repository's history, intended to "Archive TASK template contract and enforce proof-first ledger maintenance".[1] A granular inspection of the repository dump [1] confirms that the "hard" infrastructure required for this transition is present and functional. The PoW.md file has been established at the repository root, populated with a strictly schema-compliant entry for DP-OPS-0060 that captures the Packet ID, Timestamp, Work Branch, Base HEAD, and precise Receipt pointers.[1] This demonstrates that the *mechanism* for proof logging is active. Simultaneously, the docs/ops/specs/surfaces/task.md specification file now houses the canonical TASK template, bounded by `` delimiters, enabling the "Hydration" strategy where active work surfaces are generated from a secured source of truth rather than copied from potentially drifting local files.[1]

However, the audit reveals that the *removal* of legacy structures was not fully executed, resulting in a repository state that is structurally advanced but semantically conflicted. The TASK.md file, which serves as the "Contract" for active work, retains **Section 4.1: Thread Transition** and **Section 5: Work Log**.[1] These sections are artifacts of a previous governance era where the continuity of work was maintained through narrative handoffs within the task file itself. In the new "Proof-First" architecture, the TASK file is designed to be ephemeral—a "stateless routing surface" that resets completely between sessions, with all historical continuity offloaded to the persistent SoP and PoW ledgers. By retaining the Work Log, the system inadvertently validates the practice of storing state in a disposable container, directly contradicting the axiom that "Context is file-defined state, not memory".[1]

2.2 The Risk of Governance Drift in a Hybrid State

The persistence of these legacy sections poses a significant risk of "Governance Drift." In complex sociotechnical systems involving both human operators and AI agents, the interface often dictates the behavior. If the TASK.md dashboard presents a "Work Log" section, the operator is cognitively primed to utilize it, documenting their actions in an informal, narrative format that is destined for deletion when the task resets. This creates "Invisible Work"—effort and decision-making that are recorded in a temporary medium and lost upon thread transition, failing to make it into the permanent PoW or SoP records.

True "Proof-First" governance requires the elimination of these "scratchpad" surfaces. The

architecture must force the operator to commit all significant execution details to the PoW.md ledger (for forensic proof) or the RESULTS artifact (for audit verification). The removal of Section 5 is not merely a cleanup task; it is a behavioral constraint functionality designed to channel operator inputs into the verifiable, immutable ledgers that constitute the repository's "Long-Term Memory." Without this excision, the repository remains vulnerable to "Context Fatigue," where the active work surface becomes bloated with historical debris that should have been archived, degrading the signal-to-noise ratio for future agents.[1]

Feature	Legacy Model (Trust)	Target Model (Verification)	Current Status
State Persistence	TASK.md Work Log	SoP.md & PoW.md	**Hybrid / Conflicted**
Handoff Logic	"Thread Transition"	Clean Reset via Template	**Legacy Present**
Completion Proof	Narrative Sign-off	Cryptographic Entry	**Proof Enabled but Optional**
Enforcement	Human Compliance	Automated Linting/Policing	**Partial (PoW missing)**

---

3. Architectural Refactoring of the TASK Surface

3.1 Deconstructing the Legacy Sections

The current structure of TASK.md (and its template source docs/ops/specs/surfaces/task.md) includes a legacy tail that must be amputated to finalize the transition. Specifically, **Section 4.1: Thread Transition** describes a manual process of carrying over context that is now obsolete due to the "Hydration" capabilities of ops/bin/open and the "Context Snapshot" features of ops/bin/context.[1] The idea that a "Thread" needs to be "Transitioned" implies that the TASK file serves as a vessel for continuity. In the new architecture, continuity is maintained by the Map and the Ledgers, not the work surface. The TASK file should be viewed as a disposable "Execution Container" that is instantiated, utilized, and then recycled.

Section 5: Work Log is equally problematic. It serves as a linear accumulator of notes. In a high-churn repository, this log grows indefinitely until a manual intervention clears it. The new ops/bin/prune logic (discussed in Section 4) is designed to handle "Context Fatigue" by archiving ledger entries, but it cannot effectively manage an unstructured blob of text in

TASK.md. The existence of this section allows for "Lazy Logging," where an operator might jot down "Fixed the bug" in the Work Log but fail to create a detailed PoW entry with the specific commit hash and diff statistics.

3.2 Implementing the Mandatory Closing Block in Section 4.1

The replacement for these legacy sections is the **Mandatory Closing Block**, which transforms the vague "Closeout" process into a structured data entry API. The requirement is to implement this block within a newly defined **Section 4.1**, removing the ambiguity of where the closing metadata should reside.

The structure of this new section must be rigid to support future automation. By standardizing the Closing Block as **Section 4.1**, we enable downstream tools (harvesters) to reliably parse this section to generate Pull Request descriptions, changelogs, and release notes automatically. The fields within this block—Primary Commit Header, Pull Request Title, Final Squash Stub—are not merely documentation; they are the "meta-instructions" that the Integrator uses to merge the work into the immutable trunk.

Refactoring Specification:

The file docs/ops/specs/surfaces/task.md (the template source) must be edited to remove the existing text from ## 4. Closeout to the end of the file and replace it with the following schema:

4. Closeout

- Execute docs/MANUAL.md Closeout Cycle in order (Verify, Harvest, Refresh, Log, Prune).
- Update SoP.md and PoW.md with DP entries, including objective summary and verification commands run.
- Protocol order for closeout: Verify -> Generate Results -> COMMIT (Operator Only) -> Prune.
- Run prune hygiene at the end of the closeout sequence:./ops/bin/prune --dp=DP-OPS-XXXX --scrub.
- Use./ops/bin/prune --reset-task only for explicit TASK baseline reset after Work Log clear.
- Ensure the next session begins with refreshed session artifacts and matching receipts.

4.1 Mandatory Closing Block

Primary Commit Header (plaintext)

Pull Request Title (plaintext)

Pull Request Description (markdown)

Final Squash Stub (plaintext) (Must differ from #1)

Extended Technical Manifest (plaintext)

Review Conversation Starter (markdown)

This change structurally eliminates the "Work Log" and "Thread Transition" headings. By promoting the "Mandatory Closing Block" to a subsection (### 4.1), we align the document structure with the validation logic we will implement in the linters. This explicit subsectioning is a critical enabler for "Semantic Linting," allowing the system to verify not just that the block exists, but that it is correctly positioned as the final, definitive statement of the task's completion.

---

4. Operational Logic Refactoring: The prune Binary

The ops/bin/prune binary acts as the repository's garbage collector and hygiene enforcer. Its current implementation contains a "safety interlock" that is tightly coupled to the legacy TASK.md structure. This coupling presents a blocking issue for the proposed refactoring.

4.1 The Legacy Safety Interlock: ensure_task_work_log_clear

In the current version of ops/bin/prune [1], there exists a function—ensure_task_work_log_clear—designed to prevent the accidental destruction of unarchived work. This function parses TASK.md looking for the string ## 5. Work Log and checks if there is any content following the marker (No active thread). If it finds content, it aborts the --reset-task operation, forcing the operator to manually archive the log before resetting.

While well-intentioned, this logic is fundamentally incompatible with the removal of Section 5. If we delete Section 5 from the template but leave this check in the binary, ops/bin/prune --reset-task will catastrophically fail (or behave unpredictably) because it will search for a header that no longer exists. Furthermore, maintaining a safety check for a feature we intend to deprecate sends a mixed message about the repository's governance doctrine.

4.2 The New Safety Paradigm: ensure_pow_entry_exists

The deprecation of the Work Log requires a shift in the "Safety Philosophy." We are moving from "Check if the scratchpad is empty" (Negative Verification) to "Check if the official record is updated" (Positive Verification). The new safety interlock must verify that the work performed in the current Dispatch Packet (DP) has been cryptographically logged in the PoW.md ledger.

Proposed Logic:

The ensure_task_work_log_clear function must be removed and replaced with ensure_pow_entry_exists. This new function will:

1. **Detect the Active DP:** Parse TASK.md to identify the current DP-ID (e.g., DP-OPS-0060).
2. **Scan the Ledger:** Grep the PoW.md file for a matching entry (e.g., Packet ID: DP-OPS-0060).
3. **Enforce Ordering:** If the DP is active but no PoW entry exists, the script must abort the --reset-task operation with a fatal error: *"SAFETY VIOLATION: No PoW.md entry found for active DP-ID. Log proof before resetting task."*

This architectural shift ensures that the TASK surface cannot be wiped clean until the "Proof of Work" has been formally committed to the permanent record. It enforces the "Proof-First" behavior by making it a mechanical prerequisite for closing the session. This is a far more robust guarantee of data integrity than merely checking for an empty text block.

4.3 Implementing the Change in ops/bin/prune

The refactoring of ops/bin/prune involves locating the ensure_task_work_log_clear function and excising it. In its place, the new logic must utilize the existing detect_task_dp_id helper (likely already present to support other prune functions) to retrieve the context.

Technical Considerations:

- **Template Mode Exception:** The check should be skipped if the DP-ID is the generic template placeholder DP-XXXX, as this indicates no active work has been claimed.
- **Ledger Parsing:** The check against PoW.md must be robust. It should verify not just the presence of the string, but potentially the validity of the entry schema (e.g., ensuring Receipt pointers are present), though a simple ID check is a sufficient first-order gate.

By coupling the prune binary to the PoW ledger rather than the TASK log, we structurally reinforce the importance of the ledger. The tool itself becomes a teacher, guiding the operator to the correct behavior ("I can't reset because you haven't logged your proof").

---

5. Enforcement Logic Refactoring: tools/lint/task.sh

Governance in the Stela repository is not "requested compliance" but "enforced constraints".[1] The linter tools/lint/task.sh serves as the structural enforcement agent for the TASK surface. It ensures that the document adheres to the rigid schema required for machine readability and cognitive consistency.

5.1 Removing Legacy Heading Validation

The current implementation of tools/lint/task.sh enforces the presence of ## 4.1 Thread

Transition and ## 5. Work Log via a MANDATORY_HEADINGS array.[1] If the template is updated (as proposed in Section 3) but the linter is not, the automated gates (specifically repo-gates) will fail immediately, as the new TASK.md will lack the headers the old linter expects.

This creates a "Deadlock" scenario where the repository cannot be updated because the validation logic is out of sync with the proposed state. Therefore, the refactoring of the linter must occur atomically with the template update.

5.2 Enforcing the New Schema

The MANDATORY_HEADINGS array in tools/lint/task.sh must be updated to:

1. **Remove:** ## 4.1 Thread Transition
2. **Remove:** ## 5. Work Log
3. **Add:** ### 4.1 Mandatory Closing Block

Negative Enforcement Strategy:

Beyond simply checking for the *presence* of the new headers, the linter should actively check for the *absence* of the old ones. This is a "Negative Constraint" that prevents regression. If an operator accidentally restores an old version of TASK.md (e.g., from a stash or local backup), the linter should detect the presence of "Thread Transition" or "Work Log" and fail with a specific error message: *"FAIL: Legacy 'Work Log' detected. TASK.md must be migrated to the V3 schema."*

This "Immune System" approach ensures that legacy patterns are aggressively rejected, preventing "Zombie Governance" where old habits resurface in the codebase. The linter thus becomes the primary mechanism for enforcing the architectural decision to deprecate the work log.

---

6. Documentation and Human Factors: docs/MANUAL.md

While binaries and linters enforce constraints, the Manual (docs/MANUAL.md) explains the "Operator Mechanics".[1] It is the user interface for the repository's governance. The current manual contains instructions that are now dangerous or confusing in the context of the new architecture.

6.1 Deprecating the "Log Copy" Directive

The legacy "Closeout Cycle" in the manual currently instructs operators to *"Copy TASK.md Section 5 Work Log into the SoP.md DP entry"* and *"Reset TASK.md Section 5 Work Log to '(No active thread)'"*.[1] With Section 5 removed, following these instructions is impossible. More

critically, these instructions reinforce the mental model that the "Work Log" is the primary source of truth for history.

The manual must be rewritten to redirect the operator's attention to the PoW ledger and the RESULTS artifact. The "logging" action is no longer a copy-paste operation from a scratchpad; it is an act of **synthesis**.

6.2 Verifying the PoW Worker Directive

The user query specifically asks if the worker directive for PoW logging is adequately covered in Section 4 or docs/MANUAL.md given the removal of Section 5. The analysis indicates that **Section 4 (Closeout)** of the new TASK template (as proposed in Section 3.2) explicitly lists:

> "Update SoP.md and PoW.md with DP entries..."

However, the manual needs to be more explicit to fill the void left by the Work Log. The loss of the "scratchpad" might leave workers feeling they have nowhere to record "partial" or "contextual" details that don't fit into the rigid SoP structure.

Proposed Update to docs/MANUAL.md - Section 4 (Closeout Cycle):

We must redefine Step 4 (Log) to explicitly frame PoW.md as the destination for execution details.

Revised Text Recommendation:

> **4. Log**
>
> **Proof of Work (PoW) is the primary logging requirement.**
>
> - **Step 4a: Proof Assembly.** Verify that storage/handoff/DP-XXXX-RESULTS.md contains the git diff stats and verification outputs. This artifact replaces the legacy Work Log scratchpad.
> - **Step 4b: Ledger Entry.** Append a new entry to PoW.md following the strict schema. Use the RESULTS artifact as the source of truth for "Verification commands" and "Receipt pointers."
> - **Step 4c: History.** Update SoP.md with the *strategic* objective and outcome (the "Why").
> - **Note:** Do not log execution details in SoP.md; keep it high-level. Detailed execution proofs and "how-to" notes belong in the PoW.md "Notes" field or the RESULTS artifact.

This revision satisfies the requirement by explicitly mapping the "worker input" (previously trapped in Section 5) to specific, persistent destinations (PoW and RESULTS), ensuring no loss of fidelity in the transition.

---

7. GitHub Workflow Architecture: The Missing Guardian (pow_policing.yml)

The repository currently employs a robust "Policing" architecture for its canonical surfaces. sop_policing.yml guards the History Ledger, and task_policing.yml guards the Work Contract.[1] However, the PoW.md ledger—arguably the most critical surface for forensic integrity—is currently ungoverned by automation.

7.1 The Enforcement Gap

The sop_policing.yml workflow enforces that changes to *canon* (docs/governance) must be logged in SoP.md. However, it does not enforce that changes to *code* (logic/binaries) must be proven in PoW.md. This creates a vulnerability: an operator could merge a PR that modifies the codebase without generating the corresponding "Proof of Work" entry. In the "Trust" model, this was acceptable; in the "Verification" model, it is a critical failure.

Furthermore, PoW.md requires **Append-Only Hygiene**. The integrity of the ledger depends on the fact that history is never rewritten. Currently, nothing prevents a PR from modifying or deleting past PoW entries.

7.2 Specification for pow_policing.yml

To close this gap, a new workflow file .github/workflows/pow_policing.yml is required. This workflow will serve two primary functions:

1. **Mandate Proof for Code Changes:** Ensure that any modification to operational code (ops/, tools/, projects/) is accompanied by an update to PoW.md.
2. **Enforce Append-Only Integrity:** Ensure that the changes to PoW.md are strictly additive (no deletions of past entries).

Proposed Workflow Implementation:

YAML

```yaml
name: pow-ledger-policing

on:
  pull_request:
    types: [opened, synchronize, reopened]
```

```yaml
jobs:
  enforce-pow:
    runs-on: ubuntu-latest
    steps:
      - name: Enforce PoW update on Code Change
        uses: actions/github-script@v7
        with:
          script: |
            const pr = context.payload.pull_request;
            const files = await github.paginate(github.rest.pulls.listFiles, {
              owner: context.repo.owner,
              repo: context.repo.repo,
              pull_number: pr.number,
              per_page: 100,
            });
            const names = files.map(f => f.filename);

            // Definition of "Code" surfaces that require Proof of Work
            const codeChanged = names.some(n =>
              n.startsWith("ops/") ||
              n.startsWith("tools/") ||
              n.startsWith("projects/") ||
              n.startsWith(".github/")
            );

            const powTouched = names.includes("PoW.md");

            // Logic: If code changed, PoW must be touched.
            if (codeChanged && !powTouched) {
              core.setFailed(
                "Operational code changed but PoW.md was not updated. " +
                "Every code change requires a Proof of Work entry."
              );
            }

      - name: Checkout
        uses: actions/checkout@v4
        with:
          fetch-depth: 0 # Required for diffing
```

```
    - name: Verify PoW Append-Only Integrity
      if: contains(github.event.pull_request.changed_files, 'PoW.md')
      run: |
       # Fetch the base branch to compare against
       git fetch origin ${{ github.base_ref }}

       # Check diff for deleted lines in PoW.md (lines starting with '-')
       # We exclude the unified diff header (--- a/PoW.md)
       DELETIONS=$(git diff origin/${{ github.base_ref }} -- PoW.md | grep "^-" | grep -v "^---" | wc -l)

       if; then
         echo "FAIL: PoW.md modification detected deletions. The Ledger must be append-only."
         echo "Lines removed:"
         git diff origin/${{ github.base_ref }} -- PoW.md | grep "^-" | grep -v "^---"
         exit 1
       fi
       echo "OK: PoW.md integrity check passed (Append-only)."
```

This workflow introduces a new "Physical Law" to the repository: **Code = Proof**. It automates the discipline that was previously reliant on human vigilance.

---

8. Strategic Implications and Second-Order Insights

8.1 The "Amnesiac Worker" Model

The removal of "Thread Transition" and "Work Log" enforces a philosophical model I term the "Amnesiac Worker." In the legacy system, the TASK.md file served as a "long-running thread," carrying context from one session to the next. This allowed operators to rely on implicit knowledge stored in the log. By removing this memory, we force the system to become stateless.

Every new session (DP) must effectively "re-learn" the world by reading the canonical ledgers (SoP and PoW). While this increases the "startup cost" of a task (higher cognitive load at initialization), it massively increases system resilience. It eliminates "drift" caused by reliance on stale or partial context hidden in old work logs. The worker is forced to ground their reality in the immutable proofs of the ledger, not the scratchpad notes of the previous shift.

8.2 The Closing Block as a Machine Interface

The elevation of the "Mandatory Closing Block" to a distinct subsection (4.1) is not just a formatting change; it creates a **Structured Interface** for the task. TASK.md effectively

becomes a function call:

- **Function:** Execute_DP(Context)
- **Return Value:** Section 4.1 (Closing Block)

This structure paves the way for "Level 4 Automation." Future scripts could programmatically scrape Section 4.1 from closed tasks to automatically generate release notes, update semantic versioning, or even trigger deployments, treating the human-completed task as a structured data object rather than a text document.

8.3 The "Invisible Work" Paradox

One potential risk of this transition is the loss of negative data. The "Work Log" often captured attempts that failed—"Tried X, didn't work, reverted." In a strict PoW model, if code is reverted, it doesn't appear in the final diff, and thus might not be logged in PoW. This creates a survivor bias in the history.

To mitigate this, the docs/MANUAL.md update should explicitly encourage using the Notes field in PoW.md to record "Negative Proofs"—paths taken and abandoned. This preserves the heuristic value of failure without polluting the active work surface.

---

9. Conclusion and Implementation Plan (DP-OPS-0061 Candidate)

The audit confirms that DP-OPS-0060 was a successful *structural* deployment but an incomplete *operational* transition. The repository currently possesses the organs of a "Proof-First" system (PoW, Spec-Template) but retains the vestigial limbs of the "Trust" system (Work Log, Thread Transition). To resolve this hybrid state and fully realize the architectural ambition, the following actions must be executed in the next Dispatch Packet (DP-OPS-0061):

9.1 Required Execution Steps

1. **Refactor docs/ops/specs/surfaces/task.md:**
   - Delete ## 4.1 Thread Transition and ## 5. Work Log.
   - Implement the new ## 4. Closeout structure containing ### 4.1 Mandatory Closing Block.
2. **Refactor ops/bin/prune:**
   - Excise ensure_task_work_log_clear.
   - Implement ensure_pow_entry_exists to enforce ledger logging before task reset.
3. **Refactor tools/lint/task.sh:**
   - Update MANDATORY_HEADINGS to mandate ### 4.1 Mandatory Closing Block.
   - Add a "Negative Check" to fail if Work Log is detected (Legacy Guard).

4. **Refactor docs/MANUAL.md:**
    - Rewrite "Closeout Cycle" (Step 4) to direct logging to PoW.md and RESULTS verification.
5. **Create .github/workflows/pow_policing.yml:**
    - Deploy the enforce-pow job to mandate proofs for code changes and enforce append-only integrity.
6. **System Reset:**
    - Execute ops/bin/prune --reset-task to propagate the new template to the active TASK.md surface, formally ending the era of the Work Log.

By executing these changes, Stela will complete its metamorphosis into a repository governed not by the good intentions of its operators, but by the immutable laws of its cryptographic ledgers.

Works cited

1. dump-full-main-69f7b684.txt